

```
01 import openai
02 from langchain import
03     ChatOpenAI
04
05 def generar_respuesta(prompt):
06     modelo = ChatOpenAI(
07         model = "gpt-4o"
08         temperature = 0.7
09     )
10
11     respuesta = modelo.invoke(
12         prompt
13     )
14     return respuesta.content
15
16
17 if __name__ == "__main__":
18     prompt = "Explicame qué
19     es la inteligencia artificial"
20     print(generar_respuesta(prompt))
```

UNIDAD 4:

PYTHON BASICO

CURSO

IA



Docente:
Gabriel Mosso



BRAINSTORM

Rep.Argentina 786, Sunchales, Sta fe.



GabrielNMosso@GMail.com

Tel: +54 351 3733383



Lenguaje Python:

Python es un lenguaje de programación de alto nivel, interpretado y de propósito general, conocido por su sintaxis clara y legible.

Particularidades de Python:

- **Sintaxis Simple:** Python utiliza una sintaxis que enfatiza la legibilidad del código. La indentación en lugar de llaves o palabras clave para delimitar bloques de código es una característica distintiva.
- **Interpreted Language (SCRIPT):** El código Python se ejecuta directamente sin necesidad de compilación previa, lo que facilita el desarrollo y prueba rápida.
- **Tipado Dinámico:** Las variables en Python no requieren que se especifique el tipo de dato. El tipo es determinado en tiempo de ejecución.
- **Gran Biblioteca Estándar:** Viene con una extensa biblioteca estándar que proporciona herramientas para tareas comunes, como manipulación de archivos, conexión a redes, y procesamiento de datos.
- **Portabilidad:** Python es compatible con múltiples plataformas, como Windows, macOS y Linux, lo que permite que el código sea ejecutado en diferentes sistemas operativos sin modificaciones.





Lenguaje Python:

Historia:

- **1980s:** Python fue desarrollado por Guido van Rossum en los Países Bajos como un proyecto durante su tiempo libre. Empezó a trabajar en Python a finales de los años 80 como un sucesor del lenguaje ABC.
- **1991:** La primera versión pública de Python, 0.9.0, fue lanzada en febrero de 1991. Incluía muchas de las características esenciales que aún se encuentran en el lenguaje hoy en día, como clases con herencia, manejo de excepciones y tipos de datos fundamentales.
- **2000:** Python 2.0 fue lanzado, introduciendo características importantes como la recolección de basura y el soporte para Unicode.
- **2008:** Python 3.0 fue lanzado, marcando una actualización importante que no es completamente compatible con versiones anteriores. Se introdujeron mejoras significativas en el lenguaje, como la nueva sintaxis para imprimir y el manejo de cadenas de texto.





Impresión en el monitor y lectura del teclado:

En Python, las funciones principales para interactuar con el usuario a través del monitor y el teclado son `print()` e `input()`.

Funciones de Impresión y Lectura

- **`print()`**: Esta función se utiliza para mostrar información en el monitor (salida estándar). Puede imprimir texto, variables y otros objetos de Python.
- **`input()`**: Esta función se utiliza para leer una línea de entrada del teclado (entrada estándar). Devuelve la entrada del usuario como una cadena de texto

```
# Ejemplo de impresión en pantalla
print("Hola Mundo!!")

# Ejemplo de lectura desde el teclado
nombre_usuario = input("Por favor, ingresa tu nombre: ")
print("¡Hola, " + nombre_usuario + "!")
```





Definición de Variables:

Asignación Simple:

Se usa el operador de asignación = para definir una variable. No es necesario declarar el tipo de variable; Python lo deduce automáticamente.

Nombres de Variables:

Los nombres de las variables deben seguir estas reglas:

Deben comenzar con una letra (a-z, A-Z) o un guion bajo (_).

Pueden contener letras, números (0-9) y guiones bajos.

No pueden comenzar con un número.

No pueden ser palabras reservadas del lenguaje (como if, while, class, etc.).

Sensibilidad a Mayúsculas y Minúsculas:

Los nombres de las variables son sensibles a mayúsculas y minúsculas, es decir, variable y Variable se consideran diferentes.

```
variable = 10      # integer
variable = "texto" # ahora es una cadena

x = 10             # Asigna un entero a x
nombre = "Ana"     # Asigna una cadena a nombre

x = 10
x = 20 # Ahora x es 20
```



keywords in Python

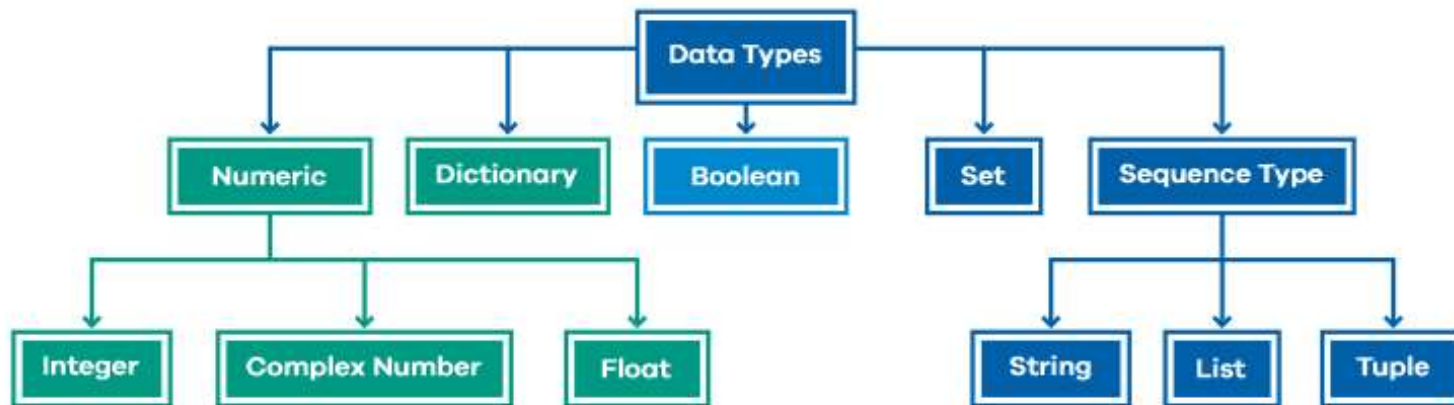
and	import	as	in
def	not	del	or
False	True	finally	try
assert	is	break	lambda
elif	pass	else	print
for	while	from	with
class	none	continue	nonlocal
except	raise	exec	return
global	yield	if	





Tipos de Variables:

Data Types in Python





Variables Numéricas:

El manejo de variables numéricas en Python es bastante flexible y sencillo. Aquí te ofrezco una descripción breve de cómo trabajar con los diferentes tipos de datos numéricos en Python:

Tipos Numéricos en Python

int (Enteros)

Representa números enteros, tanto positivos como negativos, sin decimales.

Ejemplo: $x = 5$, $y = -10$

float (Punto Flotante)

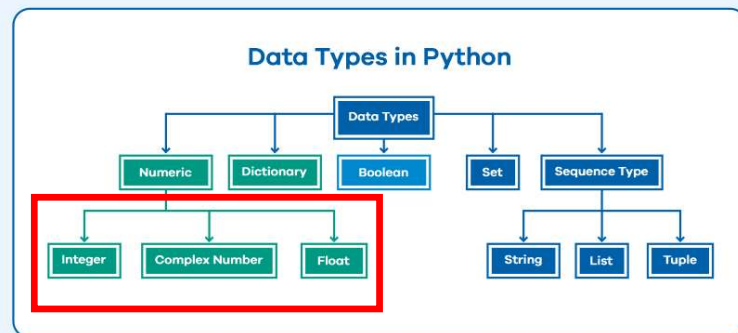
Representa números reales con decimales.

Ejemplo: $a = 3.14$, $b = -0.001$

complex (Complejos)

Representa números complejos con una parte real e imaginaria.

Ejemplo: $c = 2 + 3j$, $d = -1 - 4j$



```
entero = 10
flotante = 10.0
complejo = 10 + 1j
```





Conversión de Variables Numéricas:

La conversión de variables numéricas se refiere al proceso de cambiar el tipo de una variable de un tipo numérico a otro. Existen la conversión manual y automática:

Conversión Manual:

La conversión manual se realiza explícitamente usando funciones integradas. Aquí están las funciones más comunes para convertir entre int (enteros) y float (punto flotante):

- **int():** Convierte un número de punto flotante (float) o una cadena que representa un número entero en un entero (int). Trunca la parte decimal si es necesario.
- **float():** Convierte un número entero (int) o una cadena que representa un número decimal en un número de punto flotante (float).
- **str():** Convierte un número en una cadena (str). Aunque esto no es estrictamente una conversión entre tipos numéricos, es útil para la representación en texto.

```
numero_flotante = 3.99
numero_entero = int(numero_flotante) # Resultado: 3

numero_entero = 7
numero_flotante = float(numero_entero) # Resultado: 7.0

numero = 42
numero_como_cadena = str(numero) # Resultado: "42"

entero = 5
flotante = 3.2
resultado = entero + flotante # Resultado: 8.2, tipo: float

entero = 7
flotante = 4.5
resultado = entero * flotante # Resultado: 31.5, tipo: float

# Conversión manual
num_flotante = 9.99
num_entero = int(num_flotante) # 9

num_entero = 12
num_flotante = float(num_entero) # 12.0

# Conversión automática
num1 = 10 # int
num2 = 5.5 # float
resultado = num1 / num2 # Resultado: 1.8181818181818182,
tipo: float
```





Conversión de Variables Numéricas:

La conversión de variables numéricas se refiere al proceso de cambiar el tipo de una variable de un tipo numérico a otro. Existen la conversión manual y automática:

Conversión Automática:

Python realiza conversiones automáticas o implícitas cuando se realizan operaciones con diferentes tipos numéricos.

Promoción de Tipos: Cuando se realiza una operación entre un int y un float, Python convierte el int a float para garantizar que el resultado tenga la precisión del float.

Conversión en Operaciones: Al sumar, restar, multiplicar, o dividir números de diferentes tipos, Python convierte los operandos al tipo más general o más preciso.

```
numero_flotante = 3.99
numero_entero = int(numero_flotante) # Resultado: 3

numero_entero = 7
numero_flotante = float(numero_entero) # Resultado: 7.0

numero = 42
numero_como_cadena = str(numero) # Resultado: "42"

entero = 5
flotante = 3.2
resultado = entero + flotante # Resultado: 8.2, tipo: float

entero = 7
flotante = 4.5
resultado = entero * flotante # Resultado: 31.5, tipo: float

# Conversión manual
num_flotante = 9.99
num_entero = int(num_flotante) # 9

num_entero = 12
num_flotante = float(num_entero) # 12.0

# Conversión automática
num1 = 10 # int
num2 = 5.5 # float
resultado = num1 / num2 # Resultado: 1.8181818181818182,
tipo: float
```





Variables Booleanas (Bool):

Las variables booleanas en Python son un tipo de datos que representan valores binarios (True / False). Estos valores son muy útiles para la lógica de programación y las decisiones condicionales. Aquí te explico brevemente qué son y qué operaciones puedes realizar con ellas:

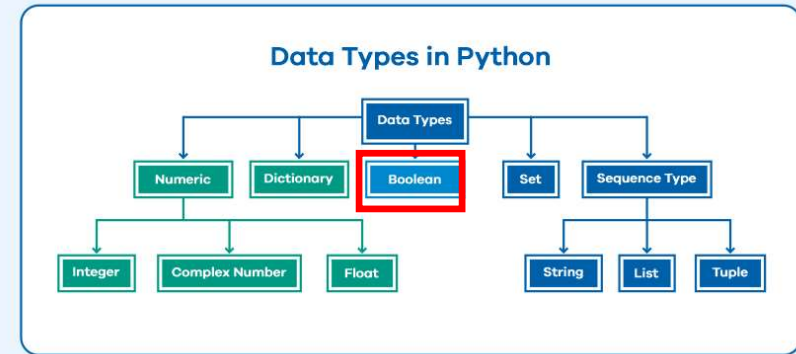
Valores Booleanos:

True: Representa la verdad o afirmación.

False: Representa la falsedad o negación.

Tipo de Dato:

El tipo de dato booleano en Python es bool. Los valores True y False son instancias de esta clase.



```
es_verdadero = True  
es_falso = False
```

```
tipo = type(es_verdadero) # Resultado: <class 'bool'>
```





Operaciones con Variables Booleanas

Operadores Lógicos:

AND (and):

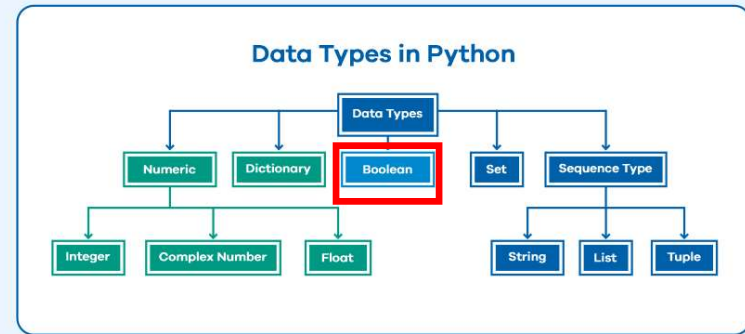
Devuelve True si ambos operandos son True, y False en cualquier otro caso.

OR (or):

Devuelve True si al menos uno de los operandos es True, y False si ambos son False.

NOT (not):

Devuelve True si el operando es False, y False si el operando es True.



AND	OR	X-OR
$0 \& 0 = 0$	$0 \mid 0 = 0$	$0 \wedge 0 = 0$
$0 \& 1 = 0$	$0 \mid 1 = 1$	$0 \wedge 1 = 1$
$1 \& 0 = 0$	$1 \mid 0 = 1$	$1 \wedge 0 = 1$
$1 \& 1 = 1$	$1 \mid 1 = 1$	$1 \wedge 1 = 0$

```
a = True  
b = False
```

```
resultado = a and b # Resultado: False
```

```
resultado = a or b # Resultado: True
```

```
resultado = not a # Resultado: False
```



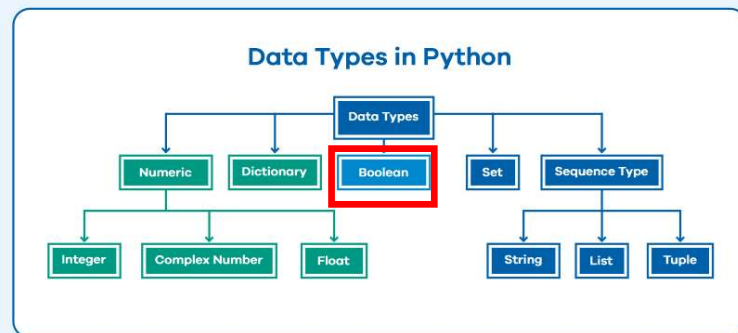


Operaciones con Variables Booleanas

Comparaciones:

Las comparaciones devuelven valores booleanos (True o False) basados en la relación entre dos valores.

- **Igualdad (==):** Devuelve True si los valores son iguales, y False en caso contrario.
- **Desigualdad (!=):** Devuelve True si los valores son diferentes, y False si son iguales.
- **Mayor que (>):** Devuelve True si el primer valor es mayor que el segundo, y False en caso contrario.
- **Menor que (<):** Devuelve True si el primer valor es menor que el segundo, y False en caso contrario.
- **Mayor o igual que (>=):** Devuelve True si el primer valor es mayor o igual que el segundo, y False en caso contrario.
- **Menor o igual que (<=):** Devuelve True si el primer valor es menor o igual que el segundo, y False en caso contrario.



```
resultado = (5 == 5) # Resultado: True
resultado = (5 != 3) # Resultado: True
resultado = (10 > 5) # Resultado: True
resultado = (3 < 5)  # Resultado: True
resultado = (5 >= 5) # Resultado: True
resultado = (3 <= 4) # Resultado: True
```

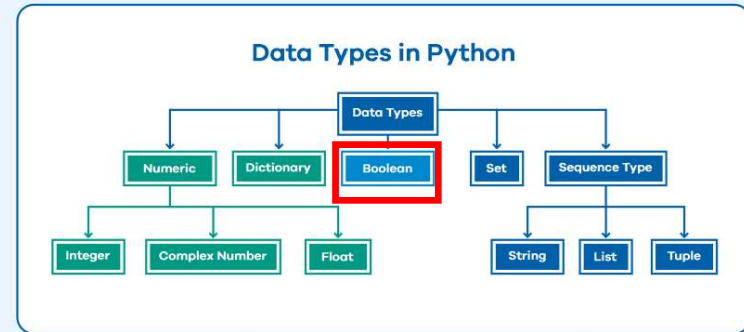




Operaciones con Variables Booleanas

Operaciones con Comparaciones y Booleanos:

Puedes combinar comparaciones y operaciones booleanas para crear expresiones lógicas más complejas.



```
a = 10  
b = 20  
c = 15
```

```
resultado = (a < b) and (c > a) # Resultado: True
```





STRING (Cadena de caracteres):

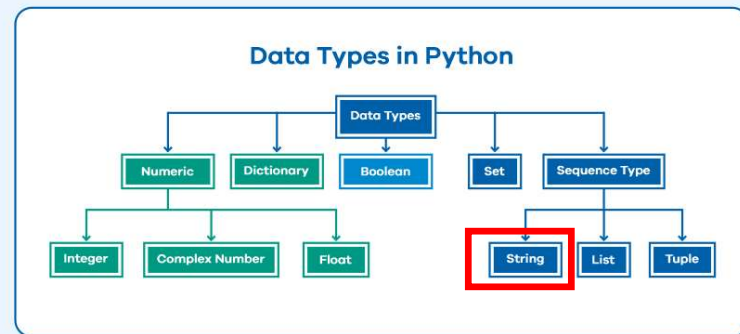
Las variables de tipo cadena (string) en Python son usadas para representar texto. Aquí tienes una breve descripción de cómo funcionan y cómo se utilizan:

Definición:

Cadena de Texto: Una cadena de texto es una secuencia de caracteres (letras, números, símbolos y espacios) **encerrada entre comillas simples (') o dobles (")**.

Características:

- **Inmutabilidad:** Las cadenas en Python son inmutables, lo que significa que una vez creadas, no pueden ser modificadas. Cualquier operación que modifique una cadena en realidad crea una nueva cadena.
- **Indexación y Slicing** Puedes acceder a caracteres individuales de una cadena utilizando índices (basados en cero) y obtener subcadenas (slicing).
- **Longitud:** La función `len()` devuelve la longitud de la cadena, es decir, el número de caracteres que contiene.
- **Concatenación:** Puedes unir cadenas utilizando el operador `+`.



```
texto = "Python"

primer_caracter = texto[0]    # Resultado: 'P'

subcadena = texto[1:4]       # Resultado: 'yth'

longitud = len(texto)        # Resultado: 6

# Operaciones comunes
concatenacion = "Python " + "es " + "genial" # "Python es genial"
repeticion = "¡Hola! " * 3 # "¡Hola! ¡Hola! ¡Hola! "
```





STRING (Cadena de caracteres):

Métodos de Cadenas:

Las cadenas tienen muchos métodos integrados para manipulación y análisis. Algunos de los más comunes son:

- **lower()**: Convierte todos los caracteres a minúsculas.
- **upper()**: Convierte todos los caracteres a mayúsculas.
- **strip()**: Elimina espacios en blanco al principio y al final de la cadena.
- **replace()**: Reemplaza todas las ocurrencias de un substring por otro.
- **split()**: Divide la cadena en una lista de subcadenas utilizando un delimitador.

Formateo: Puedes formatear cadenas para incluir variables utilizando f-strings (en Python 3.6+) o el método `format()`.

```
texto = "Python"
texto_minusculas = texto.lower() # Resultado: "python"

texto_mayusculas = texto.upper() # Resultado: "PYTHON"

texto = "  Hola  "
texto_strip = texto.strip() # Resultado: "Hola"

texto = "Hola mundo"
texto_reemplazado = texto.replace("mundo", "Python") # Resultado: "Hola Python"

texto = "Hola mundo Python"
lista = texto.split() # Resultado: ['Hola', 'mundo', 'Python']

nombre = "Ana"
edad = 30
mensaje = f"Hola, {nombre}. Tienes {edad} años." # Usando f-strings
mensaje2 = "Hola, {}. Tienes {} años.".format(nombre, edad) # Usando format()

# Formateo de cadenas
nombre = "Carlos"
mensaje = f"Hola, {nombre}!" # "Hola, Carlos!"
```





Listas (list):

Definición: Las listas son colecciones ordenadas y mutables de elementos. Puedes modificar, agregar o eliminar elementos en una lista después de su creación.

Sintaxis: Se definen usando corchetes `[]` y los elementos se separan por comas.

Características:

Mutabilidad: Puedes cambiar los elementos de una lista después de su creación.

Ordenada: Los elementos tienen un orden específico que se mantiene.

Permite duplicados: Puedes tener elementos repetidos en una lista.

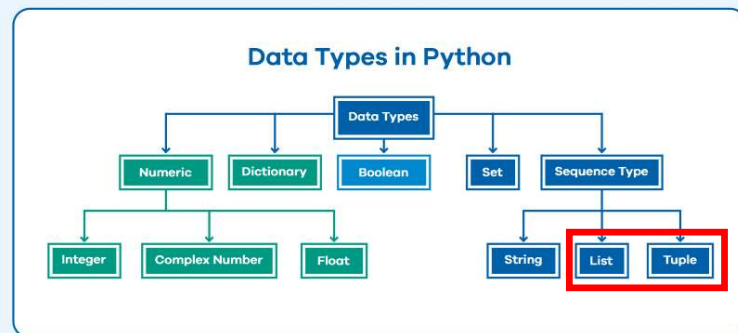
Métodos comunes:

append(): Añade un elemento al final.

remove(): Elimina el primer elemento con el valor especificado.

pop(): Elimina y devuelve un elemento en una posición específica (por defecto, el último).

extend(): Añade los elementos de una lista al final de otra lista.



```
lista = [1, 2, 3, "cuatro", 5.0]

mi_lista = [1, 2, 3, 4, 5]

mi_lista.append(6) # [1, 2, 3, 4, 5, 6]

mi_lista[0] = 10 # [10, 2, 3, 4, 5, 6]
```





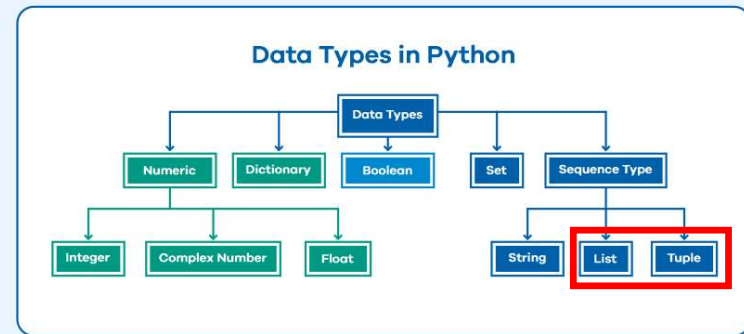
Tuplas (tuple):

Definición: Las tuplas son colecciones ordenadas e inmutables de elementos. Una vez creada, no puedes modificar, agregar ni eliminar elementos en una tupla.

Sintaxis: Se definen usando paréntesis () y los elementos se separan por comas.

Características:

- **Inmutabilidad:** No puedes cambiar los elementos de una tupla una vez que ha sido creada.
- **Ordenada:** Los elementos tienen un orden específico que se mantiene.
- **Permite duplicados:** Puedes tener elementos repetidos en una tupla.
- **Métodos comunes:** Las tuplas tienen menos métodos que las listas debido a su inmutabilidad. Algunos métodos disponibles son `count()` (cuenta las ocurrencias de un elemento) y `index()` (devuelve el índice de la primera aparición de un elemento).



```
tupla = (1, 2, 3, "cuatro", 5.0)

mi_tupla = (1, 2, 3, 4, 5)
# No se puede modificar mi_tupla directamente
```





Comparación Rápida (List vs Tuple)

En Python, las listas (list) y las tuplas (tuple) son estructuras de datos que se utilizan para almacenar colecciones de elementos. Aunque ambas sirven para almacenar múltiples elementos, tienen características y usos diferentes.

Mutabilidad:

Listas: Mutables (pueden ser modificadas).

Tuplas: Inmutables (no pueden ser modificadas).

Sintaxis:

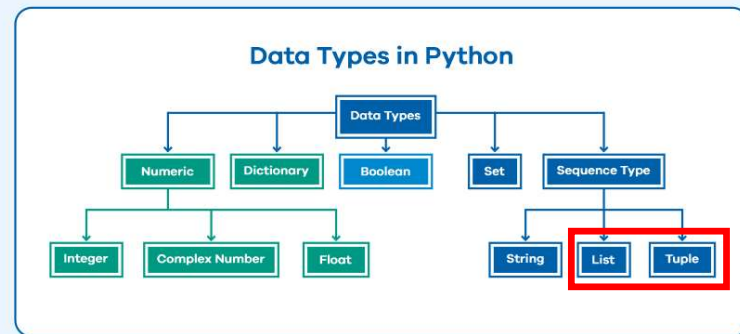
Listas: []

Tuplas: ()

Métodos:

Listas: Más métodos disponibles para modificar y gestionar los datos.

Tuplas: Menos métodos disponibles debido a la inmutabilidad.



```
# Listas
frutas = ["manzana", "banana", "cereza"]
frutas.append("naranja") # ["manzana", "banana", "cereza", "naranja"]
frutas[1] = "kiwi" # ["manzana", "kiwi", "cereza", "naranja"]

# Tuplas
puntos = (10, 20, 30)
# puntos[0] = 15 # Esto causaría un error, ya que las tuplas son inmutables
indice = puntos.index(20) # Resultado: 1
```





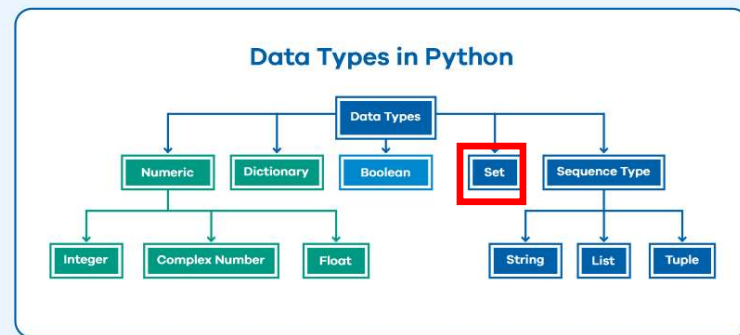
SET:

Un set en Python es una colección de elementos únicos y no ordenados. A diferencia de las listas y tuplas, los sets no mantienen un orden de los elementos y no permiten elementos duplicados.

Sintaxis: Se define usando llaves {} o la función set().

Características

- **Elementos Únicos:** Un set no puede contener elementos duplicados. Si se intenta añadir un elemento que ya existe en el set, no se añadirá.
- **Desordenado:** Los sets no mantienen un orden de los elementos. La posición de los elementos puede cambiar y no se puede acceder a los elementos por índice.
- **Mutabilidad:** Los sets son mutables, lo que significa que puedes añadir o eliminar elementos después de su creación.



```
mi_set = {1, 2, 3, 4}
```

```
otro_set = set([1, 2, 3, 4]) # También se puede  
crear un set a partir de una lista
```

```
numeros = {1, 2, 2, 3} # Resultado: {1, 2, 3}
```

```
numeros = {10, 20, 30}  
# No se puede acceder a los elementos por índice
```





SET:

Un set en Python es una colección de elementos únicos y no ordenados. A diferencia de las listas y tuplas, los sets no mantienen un orden de los elementos y no permiten elementos duplicados.

Operaciones Comunes:

Añadir Elementos: `add()`

Eliminar Elementos: `remove()` o `discard()`

Eliminar y Obtener Elemento: `pop()`

Limpieza: `clear()`

Operaciones con Sets:

Unión: `union()` o "OR"

Intersección: `intersection()` o "AND"

Diferencia: `difference()` o "-"

Diferencia Simétrica: `symmetric_difference()` o "^"

```
mi_set = {1, 2, 3}
```

```
mi_set.add(4) # Añade el elemento 4
```

```
mi_set.remove(5) # Elimina 5 del set. Lanza error si el elemento no existe
```

```
mi_set.discard(5) # Elimina 5 del set sin error si el elemento no existe
```

```
mi_set.clear() # Elimina todos los elementos del set
```

```
set1 = {1, 2, 3}
```

```
set2 = {3, 4, 5}
```

```
union = set1.union(set2) # Resultado: {1, 2, 3, 4, 5}
```

```
union = set1 | set2 # También se puede usar el operador |
```

```
interseccion = set1.intersection(set2) # Resultado: {3}
```

```
interseccion = set1 & set2 # También se puede usar el operador &
```

```
diferencia = set1.difference(set2) # Resultado: {1, 2}
```

```
diferencia = set1 - set2 # También se puede usar el operador -
```

```
# Creación de sets
```

```
frutas = {"manzana", "banana", "cereza"}
```

```
# Añadir y eliminar elementos
```

```
frutas.add("naranja") # {'manzana', 'banana', 'cereza', 'naranja'}
```

```
frutas.remove("banana") # {'manzana', 'cereza', 'naranja'}
```





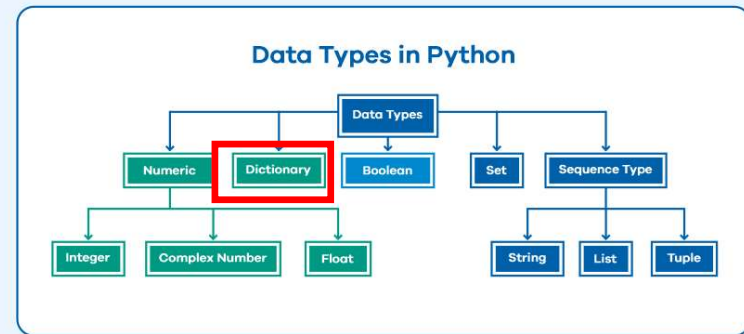
DICCIONARIO:

Un diccionario en Python es una colección de elementos donde cada elemento es un par clave-valor. A diferencia de las listas y los sets, los diccionarios permiten almacenar datos en una estructura de clave-valor, lo que facilita el acceso y manipulación de los datos mediante las claves. Aquí tienes una descripción detallada de los diccionarios:

Sintaxis: Se define usando llaves {} con pares clave-valor separados por comas. La clave y el valor se separan por dos puntos ":".

Características:

- **Clave-Valor:** Cada elemento en un diccionario es un par clave-valor. La clave actúa como un identificador único para acceder al valor asociado.
- **Desordenado:** Los diccionarios no mantienen un orden específico de los elementos. En versiones más recientes de Python (3.7+), los diccionarios conservan el orden de inserción, pero esto no debe ser dependido en aplicaciones que requieren ordenamiento garantizado.
- **Mutabilidad:** Los diccionarios son mutables. Puedes modificar, añadir o eliminar pares clave-valor después de su creación.
- **Claves Únicas:** Las claves en un diccionario deben ser únicas y deben ser de un tipo inmutable, como cadenas, números o tuplas. Los valores pueden ser de cualquier tipo de datos y pueden repetirse.



```
diccionario = {"clave1": "valor1", "clave2": "valor2"}
```

```
mi_diccionario = {  
    "nombre": "Juan",  
    "edad": 30,  
    "ciudad": "Madrid"  
}
```





DICCIONARIO:

Operaciones Comunes

Acceso a Valores: Puedes acceder al valor asociado a una clave específica usando la notación de corchetes [] o el método get().

Añadir o Modificar Elementos: Puedes añadir un nuevo par clave-valor o modificar el valor de una clave existente.

Eliminar Elementos: Puedes eliminar un par clave-valor usando el método pop() o del.

Métodos Comunes:

- **keys():** Devuelve un objeto vista de todas las claves en el diccionario.
- **values():** Devuelve un objeto vista de todos los valores en el diccionario
- **items():** Devuelve un objeto vista de todos los pares clave-valor en el diccionario.
- **clear():** Elimina todos los elementos del diccionario.
- **Comprobar Existencia:** Puedes verificar si una clave está en el diccionario utilizando el operador in.

```
nombre = mi_diccionario["nombre"] # "Juan"
edad = mi_diccionario.get("edad") # 30

mi_diccionario["pais"] = "Argentina" # Añade una nueva clave "pais"
mi_diccionario["edad"] = 31 #Modifica el valor de la clave "edad"

valor_eliminado = mi_diccionario.pop("ciudad") # Elimina la clave
"ciudad" y devuelve su valor

del mi_diccionario["nombre"] # Elimina la clave "nombre"

claves = mi_diccionario.keys() # dict_keys(['edad', 'pais'])
valores = mi_diccionario.values() # dict_values([31, 'Argentina'])

items = mi_diccionario.items() # dict_items([('edad', 31), ('pais',
'Argentina')])

mi_diccionario.clear() # {}

existe_clave = "edad" in mi_diccionario # True
```





Ejemplo: Calculadora de Números.

Este programa pide al usuario que ingrese dos números y luego le ofrece la opción de realizar una de las cuatro operaciones matemáticas básicas (suma, resta, multiplicación o división).



```
# Mostrar opciones al usuario
print("Selecciona una operación:")
print("1. Suma")
print("2. Resta")
print("3. Multiplicación")
print("4. División")

# Tomar entrada del usuario
opcion = input("Ingresa el número de la operación deseada (1/2/3/4): ")

if opcion in ['1', '2', '3', '4']: # Verificar que la opción sea válida

    num1 = float(input("Ingresa el primer número: ")) # Solicitar N1
    num2 = float(input("Ingresa el segundo número: ")) # Solicitar N2
    # Ejecutar la operación seleccionada
    if opcion == '1':
        resultado = num1 + num2
        print("La suma es: " + str(resultado))
    elif opcion == '2':
        resultado = num1 - num2
        print("La resta es: " + str(resultado))
    elif opcion == '3':
        resultado = num1 * num2
        print("La multiplicación es: " + str(resultado))
    elif opcion == '4':
        resultado = num1 / num2
        print("La división es: " + str(resultado))
else:
    print("Opción no válida. Por favor selecciona una opción entre 1 y 4.")
```

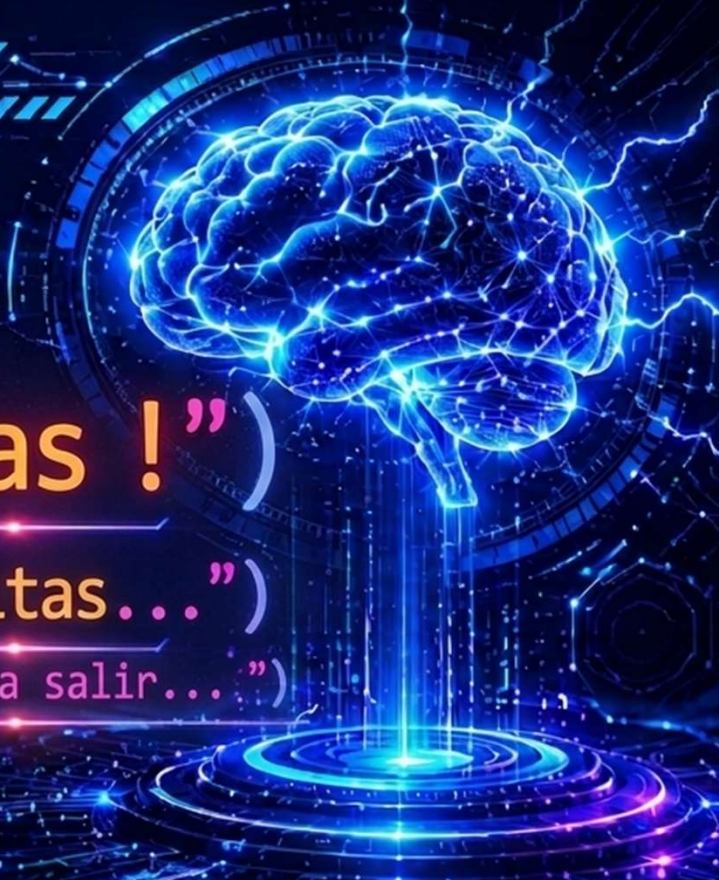



```
01 import openai
02 from IPython import
03     chatOpenAI
04
05 def generar_respuesta(prompt):
06     modelo = chatOpenAI(
07         model = "gpt-4o"
08         temperature = 0.7
09     )
10     respuesta = modelo.invoke(
11     ) prompt
12     return respuesta.content
13 )
14
15 if __name__ == "__main__":
16     print("¡Gracias!")
17     print("dudas, consultas...")
18     input("Presione una tecla para salir...")
```

> print("Gracias !")

> print("dudas, consultas...")

> input("Presione una tecla para salir...")



Docente:
Gabriel Mosso



BRAINSTORM

Rep. Argentina 786, Sunchoales, Sta fe.



GabrielNMosso@GMail.com



Tel: +54 351 3733383

